Лабораторная работа № 1

Функции получения системной информации

1. Наименование и цель лабораторной работы

Наименование: «Функции получения системной информации».

Цель работы: получение практических навыков программирования с использованием Win32 API для получения информации об операционной системе, системных параметрах, цветах интерфейса, времени и состоянии устройств ввода.

2. Постановка задачи

Разработать программу на языке C++ с использованием Win32 API, обеспечивающую получение системной информации в соответствии с вариантом 10.

Программа должна выполнять следующие действия:

1. Получать имя компьютера и имя текущего пользователя.
2. Определять пути к каталогу Windows, системному каталогу и временному каталогу.
3. Получать сведения о версии операционной системы.
4. Определять три системные метрики.
5. Получать значения трёх системных параметров.
6. Определять текущие значения системных цветов COLOR_BTNSHADOW, COLOR_INFOTEXT и COLOR_CAPTIONTEXT, а также выполнять временное изменение одного из них с последующим восстановлением исходного значения.
7. Использовать функции GetLocalTime, GetTimeZoneInformation и SetSystemTimeAdjustment.
8. Использовать дополнительные функции GetAsyncKeyState, GetKeyboardLayoutList, GetTickCount и SetLocaleInfo.

В качестве системных метрик выбраны ширина и высота экрана, а также количество подключённых мониторов. В качестве системных параметров

выбраны настройки мыши, задержка клавиатуры перед началом автоповтора и скорость автоповтора символов. Эти показатели выбраны самостоятельно, поскольку в предоставленной методичке отсутствует отдельная таблица их распределения по вариантам.

3. Описание методики и алгоритма решения задачи

Программа реализуется на языке C++ с использованием Win32 API — набора функций операционной системы Windows, предоставляющих программный доступ к системной информации и настройкам.

Алгоритм работы программы:

1. Подключить заголовочные файлы, необходимые для использования Win32 API и стандартных средств языка C++.
2. Получить имя компьютера и имя пользователя.
3. Определить пути к каталогу Windows, системному каталогу и временному каталогу.
4. Получить сведения о версии операционной системы.
5. С помощью функции GetSystemMetrics определить ширину и высоту экрана, а также количество мониторов.
6. С помощью функции SystemParametersInfo получить три выбранных системных параметра.
7. Получить значения трёх системных цветов с помощью функции GetSysColor.
8. Временно изменить цвет COLOR_BTNSHADOW с помощью SetSysColors, после чего восстановить его исходное значение.
9. Получить местные дату и время с помощью GetLocalTime.
10. Получить сведения о часовом поясе с помощью GetTimeZoneInformation.
11. Получить текущие параметры коррекции системного времени и выполнить вызов SetSystemTimeAdjustment с теми же значениями, не задавая новые параметры коррекции.

12. Проверить состояние клавиши Shift с помощью GetAsyncKeyState.
13. Получить список доступных раскладок клавиатуры с помощью GetKeyboardLayoutList.
14. Определить количество миллисекунд, прошедших с момента запуска системы, с помощью GetTickCount.
15. Получить текущий краткий формат даты и передать его функции SetLocaleInfo, чтобы продемонстрировать работу функции без намеренного изменения формата.
16. Вывести результаты выполнения программы и обработать возможные ошибки вызова API-функций.

При работе с системными настройками программа проверяет результаты вызовов функций. Для SetSystemTimeAdjustment используются текущие параметры коррекции времени, полученные посредством GetSystemTimeAdjustment. Это позволяет не задавать произвольные значения, изменяющие ход системных часов.

4. Описание использованных API-функций

GetComputerNameW — получает имя компьютера. Результат записывается в буфер, размер которого передаётся функции через параметр.

GetUserNameW — получает имя пользователя, связанного с текущим потоком.

GetWindowsDirectoryW — возвращает путь к каталогу Windows.

GetSystemDirectoryW — возвращает путь к системному каталогу Windows.

GetTempPathW — возвращает путь к каталогу временных файлов.

GetVersionExW — получает сведения о версии операционной системы в структуре OSVERSIONINFOEXW. Получаемые сведения могут зависеть от совместимости приложения с версиями Windows.

GetSystemMetrics — возвращает значения системных метрик. В программе используются:

- SM_CXSCREEN — ширина экрана в пикселях;
- SM_CYSCREEN — высота экрана в пикселях;

- SM_CMONITORS — количество мониторов.

SystemParametersInfoW — получает сведения о системных параметрах. В программе используются:

- SPI_GETMOUSE — получает параметры мыши;
- SPI_GETKEYBOARDDELAY — получает задержку перед началом автоповтора клавиатуры;
- SPI_GETKEYBOARDSPEED — получает скорость автоповтора клавиатуры.

GetSysColor — получает текущее значение системного цвета по его символьной константе. Возвращаемое значение имеет тип COLORREF.

SetSysColors — изменяет значения одного или нескольких системных цветов. В программе функция применяется для временного изменения COLOR_BTNSHADOW с последующим восстановлением исходного цвета.

GetLocalTime — получает текущие местные дату и время в структуре SYSTEMTIME.

GetTimeZoneInformation — получает сведения о часовом поясе, включая смещение времени и информацию о стандартном времени и летнем времени.

GetSystemTimeAdjustment — получает параметры коррекции системных часов и сведения о том, отключена ли периодическая коррекция.

SetSystemTimeAdjustment — включает, отключает или задаёт параметры периодической коррекции системного времени. В программе функция вызывается с текущими значениями, предварительно полученными через GetSystemTimeAdjustment. Вызов может завершиться ошибкой, например, из-за отсутствия необходимых привилегий.

GetAsyncKeyState — возвращает сведения о состоянии указанной клавиши. В программе проверяется старший бит результата, указывающий, нажата ли клавиша Shift в момент проверки.

GetKeyboardLayoutList — получает список идентификаторов раскладок клавиатуры, доступных в системе.

GetTickCount — возвращает количество миллисекунд, прошедших с момента запуска системы. Значение имеет тип DWORD и при длительной работе системы может переполняться.

GetLocaleInfoW — получает сведения о региональных настройках, в данном случае — о кратком формате даты.

SetLocaleInfoW — устанавливает региональные сведения. В программе передаётся уже полученный краткий формат даты, чтобы не вводить произвольные изменения в настройки пользователя.

5. Описание разработанной программы

Разработанная программа представляет собой консольное приложение для операционной системы Windows. Она последовательно выполняет вызовы Win32 API и выводит результаты в консоль.

Программа разделена на логические блоки: получение основных сведений о компьютере и пользователе, определение путей системных каталогов, получение версии ОС, анализ метрик и системных параметров, работа с системными цветами, получение времени и часового пояса, а также использование дополнительных API-функций варианта 10.

Для получения строковых данных используются буферы достаточного размера. Результаты вызовов, которые могут завершиться неудачно, проверяются. При ошибке выводится код, возвращённый функцией GetLastError, если это применимо к конкретному вызову.

При работе с цветами программа сохраняет исходное значение COLOR_BTNSHADOW, устанавливает временный цвет и после подтверждения пользователя восстанавливает исходное значение. Вызов SetSystemTimeAdjustment не задаёт произвольную коррекцию системного времени: используются текущие параметры, полученные от ОС. Функция SetLocaleInfoW получает текущее значение формата даты и передаёт его обратно без намеренного изменения настройки.

Для компиляции программы требуется компилятор C++ для Windows с поддержкой Win32 API, например MSVC в Visual Studio или MinGW-w64.

6. Листинг разработанной программы

```
#define NOMINMAX
#include <windows.h>
#include <Lmcons.h>
#include <iostream>
#include <iomanip>
#include <sstream>
#include <string>
#include <vector>
#include <limits>

#ifdef _MSC_VER
#pragma warning(disable: 4996)
#endif

std::wstring colorToString(COLORREF color)
{
    std::wostringstream result;
    result << L"#"
           << std::uppercase << std::hex << std::setfill(L'0')
           << std::setw(2) << static_cast<int>(GetRValue(color))
           << std::setw(2) << static_cast<int>(GetGValue(color))
           << std::setw(2) << static_cast<int>(GetBValue(color));
    return result.str();
}

void printError(const wchar_t* functionName)
{
    DWORD errorCode = GetLastError();

    std::wcout << L"Ошибка " << functionName
               << L". Код ошибки: " << errorCode << L"\n";
}

void printSystemColors()
{
    const int colorIndexes[] = {
        COLOR_BTNSHADOW,
        COLOR_INFOTEXT,
        COLOR_CAPTIONTEXT
    };

    const wchar_t* colorNames[] = {
        L"COLOR_BTNSHADOW",
        L"COLOR_INFOTEXT",
        L"COLOR_CAPTIONTEXT"
    };

    std::wcout << L"\nСистемные цвета:\n";

    for (int i = 0; i < 3; ++i)
    {
        COLORREF color = GetSysColor(colorIndexes[i]);

        std::wcout << L"  " << colorNames[i] << L": "
                   << colorToString(color) << L"\n";
```

```
    }

    COLORREF originalColor = GetSysColor(COLOR_BTNSHADOW);
    COLORREF temporaryColor = RGB(255, 0, 0);

    std::wcout << L"\nПопытка временно изменить COLOR_BTNSHADOW.\n";

    if (SetSysColors(1, &colorIndexes[0], &temporaryColor))
    {
        std::wcout << L"Вызов SetSysColors выполнен.\n"
                   << L"Исходный цвет: "
                   << colorToString(originalColor) << L"\n"
                   << L"Временный цвет: "
                   << colorToString(temporaryColor) << L"\n"
                   << L"Нажмите Enter для восстановления цвета.";

        std::wcin.get();

        if (SetSysColors(1, &colorIndexes[0], &originalColor))
        {
            std::wcout << L"\nИсходное значение восстановлено.\n";
        }
        else
        {
            printError(L"SetSysColors при восстановлении");
        }
    }
    else
    {
        printError(L"SetSysColors");
        std::wcout
            << L"Системный цвет не был изменён. "
            << L"Это может зависеть от версии Windows и её настроек.\n";
    }
}

int main()
{
    SetConsoleOutputCP(CP_UTF8);
    SetConsoleCP(CP_UTF8);

    std::wcout << L"Лабораторная работа № 1\n"
               << L"Функции получения системной информации\n"
               << L"Вариант 10\n\n";

    const DWORD bufferSize = 32768;
    std::vector<wchar_t> buffer(bufferSize, L'\0');

    // 1. Имя компьютера
    DWORD computerNameSize = bufferSize;

    if (GetComputerNameW(buffer.data(), &computerNameSize))
    {
        std::wcout << L"Имя компьютера: "
                   << buffer.data() << L"\n";
    }
    else
    {
        printError(L"GetComputerNameW");
    }

    // 2. Имя пользователя
    DWORD userNameSize = bufferSize;
```

```
    if (GetUserNameW(buffer.data(), &userNameSize))
    {
        std::wcout << L"Имя пользователя: "
                   << buffer.data() << L"\n";
    }
    else
    {
        printError(L"GetUserNameW");
    }

    // 3. Каталог Windows
    UINT windowsPathLength =
        GetWindowsDirectoryW(buffer.data(), bufferSize);

    if (windowsPathLength != 0 &&
        windowsPathLength < bufferSize)
    {
        std::wcout << L"Каталог Windows: "
                   << buffer.data() << L"\n";
    }
    else
    {
        printError(L"GetWindowsDirectoryW");
    }

    // 4. Системный каталог
    UINT systemPathLength =
        GetSystemDirectoryW(buffer.data(), bufferSize);

    if (systemPathLength != 0 &&
        systemPathLength < bufferSize)
    {
        std::wcout << L"Системный каталог: "
                   << buffer.data() << L"\n";
    }
    else
    {
        printError(L"GetSystemDirectoryW");
    }

    // 5. Временный каталог
    DWORD tempPathLength =
        GetTempPathW(bufferSize, buffer.data());

    if (tempPathLength != 0 &&
        tempPathLength < bufferSize)
    {
        std::wcout << L"Временный каталог: "
                   << buffer.data() << L"\n";
    }
    else
    {
        printError(L"GetTempPathW");
    }

    // 6. Версия операционной системы
    OSVERSIONINFOEXW version{};
    version.dwOSVersionInfoSize = sizeof(version);

    if (GetVersionExW(
            reinterpret_cast<OSVERSIONINFOW*>(&version)))
    {
        std::wcout << L"\nВерсия операционной системы: "
                   << version.dwMajorVersion << L"."
```

```
                << version.dwMinorVersion
                << L", сборка "
                << version.dwBuildNumber << L"\n";
    }
    else
    {
        printError(L"GetVersionExW");
    }

    // 7. Системные метрики
    std::wcout << L"\nСистемные метрики:\n";

    std::wcout << L"Ширина экрана: "
               << GetSystemMetrics(SM_CXSCREEN)
               << L" пикс.\n";

    std::wcout << L"Высота экрана: "
               << GetSystemMetrics(SM_CYSCREEN)
               << L" пикс.\n";

    std::wcout << L"Количество мониторов: "
               << GetSystemMetrics(SM_CMONITORS)
               << L"\n";

    // 8. Системные параметры мыши и клавиатуры
    std::wcout << L"\nСистемные параметры:\n";

    int mouseParameters[3] = {};

    if (SystemParametersInfoW(
            SPI_GETMOUSE, 0, mouseParameters, 0))
    {
        std::wcout << L"Параметры мыши: "
                   << mouseParameters[0] << L", "
                   << mouseParameters[1] << L", "
                   << mouseParameters[2] << L"\n";
    }
    else
    {
        printError(L"SystemParametersInfoW (SPI_GETMOUSE)");
    }

    int keyboardDelay = 0;

    if (SystemParametersInfoW(
            SPI_GETKEYBOARDDELAY, 0, &keyboardDelay, 0))
    {
        std::wcout << L"Задержка автоповтора клавиатуры: "
                   << keyboardDelay << L"\n";
    }
    else
    {
        printError(
            L"SystemParametersInfoW (SPI_GETKEYBOARDDELAY)");
    }

    int keyboardSpeed = 0;

    if (SystemParametersInfoW(
            SPI_GETKEYBOARDSPEED, 0, &keyboardSpeed, 0))
    {
        std::wcout << L"Скорость автоповтора клавиатуры: "
                   << keyboardSpeed << L"\n";
    }
```

```
    else
    {
        printError(
            L"SystemParametersInfoW (SPI_GETKEYBOARDSPEED)");
    }

    // 9. Системные цвета и временное изменение одного из них
    printSystemColors();

    // 10. Местное время
    SYSTEMTIME localTime{};
    GetLocalTime(&localTime);

    std::wcout << L"\nМестные дата и время: "
               << std::setfill(L'0')
               << std::setw(2) << localTime.wDay << L"."
               << std::setw(2) << localTime.wMonth << L"."
               << localTime.wYear << L" "
               << std::setw(2) << localTime.wHour << L":"
               << std::setw(2) << localTime.wMinute << L":"
               << std::setw(2) << localTime.wSecond
               << std::setfill(L' ') << L"\n";

    // 11. Информация о часовом поясе
    TIME_ZONE_INFORMATION timeZone{};
    DWORD timeZoneStatus = GetTimeZoneInformation(&timeZone);

    if (timeZoneStatus != TIME_ZONE_ID_INVALID)
    {
        std::wcout << L"\nЧасовой пояс:\n"
                   << L"Стандартное название: "
                   << timeZone.StandardName << L"\n"
                   << L"Смещение Bias, минут: "
                   << timeZone.Bias << L"\n"
                   << L"Смещение стандартного времени, минут: "
                   << timeZone.StandardBias << L"\n"
                   << L"Смещение летнего времени, минут: "
                   << timeZone.DaylightBias << L"\n";

        if (timeZoneStatus == TIME_ZONE_ID_STANDARD)
        {
            std::wcout << L"Режим: стандартное время\n";
        }
        else if (timeZoneStatus == TIME_ZONE_ID_DAYLIGHT)
        {
            std::wcout << L"Режим: летнее время\n";
        }
        else
        {
            std::wcout
                << L"Режим: переход на летнее время не определён\n";
        }
    }
    else
    {
        printError(L"GetTimeZoneInformation");
    }

    // 12. Получение и повторная передача текущих параметров коррекции
    DWORD timeAdjustment = 0;
    DWORD timeIncrement = 0;
    BOOL adjustmentDisabled = FALSE;

    std::wcout << L"\nКоррекция системного времени:\n";
```

```
if (GetSystemTimeAdjustment(
        &timeAdjustment,
        &timeIncrement,
        &adjustmentDisabled))
{
    std::wcout << L"Текущая коррекция: "
               << timeAdjustment << L"\n"
               << L"Приращение системного времени: "
               << timeIncrement << L"\n"
               << L"Коррекция отключена: "
               << (adjustmentDisabled ? L"да" : L"нет")
               << L"\n";

    if (SetSystemTimeAdjustment(
            timeAdjustment, adjustmentDisabled))
    {
        std::wcout
            << L"SetSystemTimeAdjustment вызвана "
            << L"с текущими параметрами.\n";
    }
    else
    {
        printError(L"SetSystemTimeAdjustment");
        std::wcout
            << L"Вызов мог быть отклонён из-за отсутствия "
            << L"необходимых привилегий.\n";
    }
}
else
{
    printError(L"GetSystemTimeAdjustment");
}

// 13. Состояние клавиши Shift
SHORT keyState = GetAsyncKeyState(VK_SHIFT);

std::wcout << L"\nСостояние клавиши Shift: "
           << ((keyState & 0x8000) ? L"нажата" : L"не нажата")
           << L"\n";

// 14. Список раскладок клавиатуры
int layoutCount = GetKeyboardLayoutList(0, nullptr);

std::wcout << L"\nКоличество раскладок клавиатуры: "
           << layoutCount << L"\n";

if (layoutCount > 0)
{
    std::vector<HKL> layouts(layoutCount);

    int receivedCount =
        GetKeyboardLayoutList(layoutCount, layouts.data());

    if (receivedCount > 0)
    {
        for (int i = 0; i < receivedCount; ++i)
        {
            ULONG_PTR layoutValue =
                reinterpret_cast<ULONG_PTR>(layouts[i]);

            std::wcout << L"Раскладка " << (i + 1)
                       << L": 0x"
                       << std::hex << std::uppercase
```

```
                            << layoutValue << std::dec << L"\n";
            }
        }
        else
        {
            printError(L"GetKeyboardLayoutList");
        }
    }

    // 15. Количество миллисекунд с момента запуска системы
    DWORD tickCount = GetTickCount();

    std::wcout << L"\nВремя работы системы в миллисекундах: "
               << tickCount << L"\n";

    // 16. Получение и повторная запись текущего формата даты
    wchar_t dateFormatBuffer[256] = {};

    int dateFormatLength = GetLocaleInfoW(
        LOCALE_USER_DEFAULT,
        LOCALE_SSHORTDATE,
        dateFormatBuffer,
        static_cast<int>(std::size(dateFormatBuffer)));

    if (dateFormatLength > 0)
    {
        std::wstring dateFormat(dateFormatBuffer);

        std::wcout << L"\nКраткий формат даты: "
                   << dateFormat << L"\n";

        if (SetLocaleInfoW(
                LOCALE_USER_DEFAULT,
                LOCALE_SSHORTDATE,
                dateFormat.c_str()))
        {
            std::wcout
                << L"SetLocaleInfoW вызвана с текущим "
                << L"значением формата даты.\n";
        }
        else
        {
            printError(L"SetLocaleInfoW");
            std::wcout
                << L"Система не разрешила установить значение. "
                << L"Настройка не была намеренно изменена.\n";
        }
    }
    else
    {
        printError(L"GetLocaleInfoW");
    }

    std::wcout << L"\nРабота программы завершена.\n";
    return 0;
}
```

7. Результаты выполнения программы

Результаты работы программы представлены в табличной форме. Значения, зависящие от конкретного компьютера и настроек операционной системы, определяются непосредственно при запуске программы.

Таблица 1 — Результаты получения системной информации

Проверяемый параметр	Используемая функция	Результат
Имя компьютера	GetComputerNameW	Имя компьютера, заданное в Windows
Имя пользователя	GetUserNameW	Имя текущего пользователя
Путь к каталогу Windows	GetWindowsDirectoryW	Путь к каталогу Windows
Путь к системному каталогу	GetSystemDirectoryW	Путь к системному каталогу
Путь к временному каталогу	GetTempPathW	Путь к каталогу временных файлов
Версия операционной системы	GetVersionExW	Основная и дополнительная версии, номер сборки
Ширина экрана	GetSystemMetrics	Значение в пикселях
Высота экрана	GetSystemMetrics	Значение в пикселях
Количество мониторов	GetSystemMetrics	Количество мониторов, подключённых к рабочему столу
Параметры мыши	SystemParametersInfoW	Три значения параметров мыши
Задержка автоповтора клавиатуры	SystemParametersInfoW	Значение системной настройки

Скорость автоповтора клавиатуры	SystemParametersInfoW	Значение системной настройки

Таблица 2 — Результаты работы с системными цветами

Символьная константа	Используемая функция	Получаемые сведения
COLOR_BTNSHADOW	GetSysColor	Текущее значение цвета тени кнопок
COLOR_INFOTEXT	GetSysColor	Текущее значение цвета текста информационных подсказок
COLOR_CAPTIONTEXT	GetSysColor	Текущее значение цвета текста заголовка окна

Для изменения цвета используется функция SetSysColors. Программа сохраняет исходное значение COLOR_BTNSHADOW, устанавливает временное значение и затем пытается восстановить исходное. Успешность изменения и восстановления определяется по результатам выполнения соответствующих вызовов.

Таблица 3 — Результаты работы с функциями времени

Функция	Получаемые сведения или выполняемое действие
GetLocalTime	Получение текущей местной даты и времени
GetTimeZoneInformation	Получение сведений о часовом поясе и режиме времени
GetSystemTimeAdjustment	Получение текущих параметров коррекции системного времени

SetSystemTimeAdjustment	Передача текущих параметров коррекции системного времени

Таблица 4 — Результаты работы с дополнительными API-функциями варианта 10

Функция	Назначение
GetAsyncKeyState	Проверка состояния клавиши Shift
GetKeyboardLayoutList	Получение списка раскладок клавиатуры
GetTickCount	Получение количества миллисекунд, прошедших с момента запуска системы
GetLocaleInfoW	Получение текущего краткого формата даты
SetLocaleInfoW	Попытка передать системе текущее значение краткого формата даты

Таким образом, программа предусматривает получение основных системных сведений, исследование системных метрик и параметров, работу с цветами интерфейса, временем, раскладками клавиатуры и региональными настройками. Фактические значения зависят от конфигурации компьютера, а успешность операций изменения определяется операционной системой.

8. Заключение

В ходе лабораторной работы была разработана программа на языке C++ с использованием Win32 API для получения системной информации в операционной системе Windows.

В программе предусмотрено получение имени компьютера и пользователя, путей к системным каталогам, сведений о версии ОС, системных метрик и параметров. Также реализованы обращения к функциям работы с системными цветами, местным временем, часовым поясом, коррекцией системного времени, состоянием клавиатуры и региональными настройками.

В результате выполнения работы изучены принципы использования Win32 API, передачи параметров системным функциям, получения информации из структур данных и обработки ошибок при обращении к функциям операционной системы.